

HydroSense-Kenya: A Scientific Computing System for Smart Irrigation

Final Project Report — Course: ICS 2207 Scientific Computing

Semester:	February - May 2026
Project Title:	HydroSense-Kenya
Team Members:	Andrew Chweya Okero Mokandu (SCT 211-0251/2024) — <i>Reports, Slides and Source codes.</i> Griffins Koli (SCT211-0349/2024) — <i>Notebooks, Source Code</i> Bethuel Kipruto (SCT211-0387/2024) — <i>Tests, Source Code</i>
Institution:	Jomo Kenyatta University of Agriculture and Technology (JKUAT)
Date:	June 19, 2026

Executive Summary

HydroSense-Kenya is a comprehensive scientific computing system developed to address water-use efficiency challenges in Kenyan agricultural contexts. The project implements a complete workflow from data acquisition through numerical modeling, simulation, optimization, and decision support for smart irrigation management. Leveraging Python's scientific computing ecosystem, the system processes real-world sensor data, implements core numerical methods from scratch, simulates soil moisture dynamics under uncertainty, and generates optimized irrigation recommendations. The project follows a six-level incremental design, demonstrating proficiency in computational thinking, numerical methods, data science, simulation, and responsible AI-assisted programming.

1. Introduction and Problem Framing

1.1 Scientific Problem Statement

Water scarcity poses a critical challenge to agricultural productivity in Kenya, particularly for smallholder farmers and agricultural innovation hubs. Climate variability, inefficient irrigation practices, and limited access to real-time decision support tools exacerbate this challenge, leading to crop stress, reduced yields, and water wastage. The central scientific question addressed by HydroSense-Kenya is:

"Given weather and soil-sensor data, how can we model water availability, estimate water deficit, simulate future soil moisture, and recommend an efficient irrigation plan that minimizes water use without exposing crops to moisture stress?"

1.2 Computational Problem Translation

We translated this agricultural challenge into a computational problem requiring:

1. **Data Processing:** Cleaning and analyzing weather, soil moisture, and crop parameter datasets.
2. **Mathematical Modeling:** Implementing the water balance equation with evapotranspiration estimation.
3. **Numerical Methods:** Root finding for irrigation amount calculation, differentiation for moisture change rates, integration for cumulative deficits, and linear algebra for multi-zone allocation.
4. **Simulation:** Modeling soil moisture dynamics using differential equations with uncertainty quantification.
5. **Optimization:** Minimizing irrigation water while respecting moisture constraints.
6. **Visualization:** Creating interpretable scientific figures for decision support.

1.3 Project Context and Significance

The project was developed for a university demonstration farm context, representing a typical Kenyan agricultural innovation scenario. The synthetic datasets provided realistic challenges including missing values, sensor anomalies, and outliers, mirroring real-world data quality issues. This approach ensured that our solution would be applicable to actual smart irrigation systems being deployed across Kenya's agricultural sector.

1.4 Assumptions and Limitations

Key Assumptions:

- Daily time steps are sufficient for modeling soil moisture dynamics.
- The simplified evapotranspiration formula ($ET = \max(0, 0.12 \times T + 0.35 \times W + 2.4 \times \text{Solar} - 0.025 \times H)$) adequately represents crop water loss for teaching purposes.
- Soil moisture behaves as a single-layer bucket model.
- Irrigation can be applied at the beginning of each day.
- Weather patterns follow the observed historical distributions.

Limitations:

- Does not account for crop growth stage variations in water requirements.
- Simplified drainage model without considering soil texture heterogeneity.
- No groundwater table interaction modeling.

- Daily aggregation may miss sub-daily rainfall intensity effects.
- Monte Carlo simulations assume independent rainfall events.

2. Mathematical Model and Numerical Methods

2.1 Core Water Balance Model

The foundation of our system is the discrete water balance equation:

$$S(t+1) = S(t) + R(t) + I(t) - ET(t) - D(t)$$

Where:

- $S(t)$ = Soil water storage/moisture at time t
- $R(t)$ = Rainfall contribution at time t
- $I(t)$ = Irrigation applied at time t
- $ET(t)$ = Evapotranspiration (crop water loss)
- $D(t)$ = Drainage beyond field capacity

2.2 Evapotranspiration Estimation

We implemented the simplified empirical formula:

$$ET = \max(0, 0.12 \times T + 0.35 \times W + 2.4 \times \text{Solar} - 0.025 \times H)$$

Where T is Temperature (°C), W is Wind speed (m/s), Solar is Solar intensity index, and H is Humidity (%).

2.3 Numerical Methods Implemented

2.3.1 Root Finding Methods

We implemented three root-finding algorithms to determine the irrigation amount needed to reach target soil moisture:

1. **Bisection Method:** Robust but slow, guaranteed convergence.
2. **Newton-Raphson Method:** Fast convergence but requires derivative calculation.
3. **Secant Method:** Derivative-free alternative with super-linear convergence.

Performance comparison showed Newton-Raphson converged in 4-6 iterations compared to 12-15 for bisection, though it required careful handling near inflection points.

2.3.2 Numerical Differentiation

Forward, backward, and central difference approximations were implemented to estimate rates of soil moisture change:

- **Forward:** $f'(x) \approx (f(x+h) - f(x)) / h$
- **Backward:** $f'(x) \approx (f(x) - f(x-h)) / h$
- **Central:** $f'(x) \approx (f(x+h) - f(x-h)) / (2h)$

Central difference provided superior accuracy ($O(h^2)$ vs $O(h)$) and was used for daily moisture change rates.

2.3.3 Numerical Integration

Cumulative water deficits were estimated using:

- **Trapezoidal Rule:** Piecewise linear approximation.
- **Simpson's Rule:** Piecewise quadratic approximation.

Simpson's method showed lower error for smooth moisture curves, with relative errors below 0.5% compared to analytical solutions.

2.3.4 Linear Systems for Multi-Zone Allocation

A three-zone water allocation problem was solved as $Ax = b$ using Gaussian elimination with partial pivoting, enabling simultaneous optimization of water distribution across tomato, kale, and maize zones.

2.4 Differential Equation Solvers

Continuous soil moisture dynamics were modeled using:

1. **Euler Method:** Simple explicit method with $O(h)$ accuracy.
2. **Runge-Kutta Method:** Fourth-order method with $O(h^4)$ accuracy.

The RK4 method demonstrated superior stability for the stiff moisture balance equations, maintaining physical plausibility (non-negative moisture) even with larger time steps.

3. Data Processing and Analysis

3.1 Dataset Overview

Weather Data (weather_daily.csv): March 1-30, 2026. Variables: Date, rainfall (mm), temperature (°C), humidity (%), wind speed (m/s), solar index. Data quality issues: 1 missing value (March 8 rainfall), extreme values (85.0 mm on March 26, temperature 27.4°C).

Soil Sensor Data (soil_sensor_data.csv): Zones A (tomatoes), B (kale), C (maize). Variables: Timestamp, zone_id, soil moisture (%), tank level (liters), pump flow (LPM), pump power (watts), sensor status. Data quality issues: 2 missing soil moisture values (Zone_B on March 6 and 21), 1 anomalous high value (tank level 9900 on March 14), 1 sensor status "CHECK" (Zone_B on March 21), 1 outlier (soil moisture 8.5% in Zone_B on March 25).

Crop Parameters (crop_zone_parameters.csv): Parameters include Area (m²), minimum moisture threshold (%), target moisture (%), field capacity (%), and drainage coefficient.

3.2 Data Cleaning Procedures

Our cleaning decisions included:

- **Missing Values (Weather):** Linear interpolation for March 8 rainfall using neighboring day averages.
- **Missing Values (Soil):** Forward-fill for Zone_B soil moisture on March 6; March 21 value was retained as sensor "CHECK" and validated against weather patterns.
- **Outlier Detection and Treatment:**
 - *Tank Level (9900L):* Identified as sensor error on March 14; replaced with moving average of surrounding days (≈4246L).
 - *Soil Moisture (8.5%):* March 25 Zone_B anomalous low value; cross-validated with weather and other zones, flagged as outlier but retained for analysis after considering potential sensor drift.
- **Sensor Status:** "CHECK" flag on March 21 maintained in dataset; corresponding moisture value used with confidence weighting.
- **Unit Consistency:** All variables verified for consistent units throughout.

3.3 Descriptive Statistics

VARIABLE	MEAN	STD DEV	MIN	MAX	MISSING
Rainfall (mm)	11.4	17.8	0	85.0	1
Temperature (°C)	24.7	1.7	21.4	27.4	0
Humidity (%)	65.3	4.1	56.3	74.6	1
Soil Moisture A (%)	28.2	3.7	22.6	36.1	0
Soil Moisture B (%)	29.7	5.2	8.5	36.0	2
Soil Moisture C (%)	24.5	3.4	18.6	31.0	0

3.4 Scientific Visualizations & Interpretations

We created five primary visualizations to decode system relationships:

- **Time Series of Rainfall and Soil Moisture:** Dual-axis plot showing correlation between rainfall events and soil moisture response across zones. Rainfall events >20mm produced measurable soil moisture increases lasting 2-3 days.
- **Zone Comparison Box Plot:** Moisture distributions by crop zone, revealing that Zone_C (maize) consistently showed the lowest moisture levels due to its higher drainage coefficient (0.22).
- **Evapotranspiration Heatmap:** Shows ET variation across temperature-humidity space, highlighting critical spikes during high temperature and low humidity stretches.
- **Water Balance Components Stacked Bar Chart:** Daily visualization of inputs vs outputs, clearly showing that a mid-March dry spell (March 13-17) created critical stress conditions across all zones.
- **Pump Performance Scatter Plot:** Pump flow vs power consumption, identifying optimal operational efficiency ranges.

4. Simulation and Optimization Results

4.1 Soil Moisture Simulation

Euler Method Results (30-day simulation): Initial moisture: 33.2% (Zone_A), Final moisture: 24.2%, Minimum moisture: 22.6% on March 28. It took 27 days to reach minimum thresholds, prompting 8 days of mandatory irrigation events to sustain the crop.

Runge-Kutta Comparison: RK4 produced significantly smoother moisture trajectories (less oscillatory) with a maximum divergence between methods of 2.3% moisture. Euler exhibited numerical instability during rapid rainfall peaks; RK4 is highly recommended for production deployment.

4.2 Monte Carlo Uncertainty Analysis

We performed 10,000 Monte Carlo simulations with uncertain rainfall inputs:

- **Probability of water shortage (moisture < 22%):** 34%
- **Probability of over-irrigation (>40% moisture):** 12%
- **Expected irrigation demand:** 142,500 liters / 30 days
- **Worst-case demand (95th percentile):** 231,000 liters / 30 days
- **Parameter Sensitivity:** Rainfall (68% of variance), Evapotranspiration (22% of variance).

4.3 Optimized Irrigation Schedule

The optimization algorithm (gradient descent) minimized water use subject to moisture constraints:

- **Total water applied:** 124,800 liters (a 12% absolute reduction from baseline).
- **Irrigation events:** 12 of 30 days. Max irrigation occurred in Zone_C (maize).
- **Pumping time:** Reduced by 18% compared to the fixed schedule.
- **Trade-off Analysis:** More frequent, smaller applications (daily/near-daily) performed significantly better than infrequent large water dumps, preventing excessive drainage and maintaining ideal soil physics.

5. AI-Assisted Programming and Validation

We utilized AI-assisted tools for automated test generation, docstring standardization, and visualization advice. All outputs were verified using strict validation logic:

- **Test Case Generation:** Generated 5 pytest cases for the bisection method. Adjusted convergence tolerances manually from $1e-6$ to $1e-5$ and successfully cross-validated with `scipy.optimize.bisect`.
- **Code Optimization:** NumPy vectorization was used to eliminate slow loops inside the ET calculation step, accelerating execution by 3.2x.

6. Code Architecture and Implementation

The system follows a clean modular architecture separated into data, notebooks, source code (`src/`), and unit tests (`tests/`). Over 65 unit and integration tests are executed via `pytest`, achieving an excellent **94% code coverage** metric.

Floating Point Error Note

Numerical safety measures were put in place to handle floating point limitations (such as `0.1 + 0.2 != 0.3`). All logical equality evaluations utilize tolerance-based boundaries ($1e-5$) rather than direct operators to eliminate error propagation risk.

7. Results and Discussion

The results successfully validate precision irrigation engineering. The optimized schedule maintained all crop zones within their ideal targeted moisture ranges while reducing water consumption by 12%. This translates into a savings of approximately 17,000 liters over a 30-day window for this specific demonstration asset. Rainfall uncertainty remains the single largest risk element, highlighting that real-time predictive adjustments are superior to any static blueprint schedule.

8. Conclusion

HydroSense-Kenya successfully establishes an end-to-end scientific computing framework for sustainable agricultural management. By matching custom-built mathematical solvers with robust software practices, the team achieved tangible water savings while safeguarding crop health, proving the value of computational thinking in solving modern climate resilience problems.